Arbeitsweise mit KI / claude.ai

claude.ai
im Browser

- Claude einrichten
- Kurze Tour
- Erster gemeinsamer Prompt

Theorie
Sprachmodell (LLM) und claude.ai

Benjamin Thomsen

Sprachmodell (LLM)

Wie geht es vor?

- Basiert auf statistischen Mustern
- Vervollständigt Wort für Wort
- Simuliert Gedankenwege

Sprachmodell (LLM)

Was macht es nicht?

- Es schlägt nichts nach
- Es merkt nicht zuverlässig, wenn es falsch liegt (prüft nicht)
- Es führt euren Code im Hintergrund nicht aus

Sprachmodell (LLM)

Richtig und Falsch sehen identisch aus!

- **Beide klingen plausibel** – saubere Sprache, strukturierter Code
- **Beide klingen sicher** – kein Zögern, kein Vokabular der Unsicherheit
- **Kein eingebautes Warnsystem** – das Modell meldet keine Fehler, wenn es falsch liegt

Sprachmodell (LLM)

Gleiche Frage, verschiedene Antworten

- **Klarer Favorit** → fast immer dasselbe Wort (z. B. „Guten“ —> „Tag“)
- **Mehrere gute Optionen** → Die Prise Zufall entscheidet
- **Kaskadeneffekt** —> Jede Abzweigung lenkt den weiteren Verlauf um

Prompt
→ kein Determinismus

```
node projektion.js   → immer dasselbe. Garantiert.

derselbe Prompt      → oft dasselbe, manchmal nicht. Ohne Garantie.
```

Vom Coder zum Reviewer

Bewerten: Welche Lösung ist die beste?

Hinterfragen: Ist das Ergebnis fachlich richtig?

Verifizieren: Tut der Code wirklich das, was er soll?

Sprachmodell (LLM) und App (claude.ai)

Sprachmodell (z. B. Opus)

Generiert die Antwort

App (claude.ai)

Gibt ihm Werkzeuge: Websuche, Code ausführen, Chat-Suche

Zusammenspiel Modell und Werkzeug

Prompt „Wie viele Einwohner hatte Schwäbisch Gmünd 1900?“

→ Modell kommt mit Textvorhersage nicht weiter

→ Modell schreibt einen Befehl:
Websuche „Einwohner in Schwäbisch Gmünd 1900?“

→ App führt ihn aus

→ Ergebnis geht als Text zurück an das Modell

→ Modell schreibt die finale Antwort

→ Wir erhalten die Antwort vom Sprachmodell

Demo

Schaut euch den Verlauf an

- Werkzeug sichtbar (Web durchsucht, Code)
  → die App hat etwas abgerufen oder ausgeführt
- Kein Werkzeug → alles ist vorhergesagter Text

Prompt „Wie weit ist Gmünd von Stuttgart weg?“

Demo

Kontextfenster

Was beinhaltet das Kontextfenster?

- Euer Prompt (die neue Nachricht)
- Bisheriger Gesprächsverlauf (Chat-Historie)
- Ergebnisse von Werkzeugen und Hochgeladene Dateien
- Vom Modell generierte Antwort (Ausgabe-Tokens)
- Systemprompt: Anweisungen der App und eure eigenen

Kontextfenster

Regel: Neue Aufgabe, neues Gespräch

Lange Gespräche werden schlechter, nicht besser

- Verworfene Ansätze
- Alte Fehlermeldungen
- Widersprüchliche Anweisungen

Kontextfenster

Regel: Teile dem Modell mit, was es wissen soll

z. B. So sehen meine Orte aus

```
{
  "id": "gmuend",
  "name": "Schwäbisch Gmünd",
  "lat": 48.8,
  "lon": 9.79
}
```

Mit Claude Code steht das im Projekt: `CLAUDE.md`

Halluzinieren

Es erfindet, was gut ins Bild passt

- Funktionen und Parameter, die es so nicht gibt
- Bibliotheken, die es nie gab
- Code für veraltete Versionen
- Fakten, Zahlen und Quellen, die so nicht stimmen

Vibe-Coding

Prompt rein → nehmen, was läuft → weitermachen

- Geht schnell
- Oft Zielführend

Aber

- **Wenn der Code kaputtgeht, könnt ihr ihn womöglich nicht fixen.**
- **Wenn es nicht stimmt, merkt ihr es vielleicht nicht**

Ab heute: Junior-Vibe-Engineer

- Ihr müsst es bedienen können
- Ihr müsst prüfen können, was rauskommt
- **Ihr müsst erklären können, was rauskommt**

Senior-Vibe-Engineer kommt im 4. Semester

Drei goldenen Regeln

1. Spezifikation vor Code
2. Eine Änderung pro Schritt
3. Lesen und testen, was rauskam

Drei goldenen Regeln

1. Spezifikation vor Code
2. Eine Änderung pro Schritt
3. Lesen und testen, was rauskam

Drei goldenen Regeln

1. Spezifikation vor Code
2. Eine Änderung pro Schritt
3. Lesen und testen, was rauskam

KI fragen oder selbst tippen?

KI fragen (schneller, unwissend)
Formeln, die ihr nicht kennt
Fehlermeldungen, die ihr nie gesehen habt
Bibliotheken, die ihr nicht kennt
Code, den ihr schon zehnmal geschrieben habt

Selbst tippen (schneller, sicherer)
Wenn ihr genau wisst, was ihr wollt und wo ihr es ändern müsst
Wenn schneller geht es händisch zu ändern

Werkstatt

Wieviel KM liegen zwischen Gmünd und Stuttgart?

Teil 1
Prompt schreiben
Antwort und Prompt vergleichen

Teil 2
Ergebnis überprüfen

Benjamin Thomsen

Werkstatt

Teil 1 – Prompt schreiben und vergleichen

Aufgabe: Funktion die Entfernung zwischen zwei Orten auf der Erde berechnet.

- Eigenen Prompt schreiben, den Satz nicht kopieren
- Genau spezifizieren
  - Was tut die Funktion?
  - Welche Parameter haben wir?
  - Was soll herauskommen?
- Antwort und Prompt im Paar vergleichen

Werkstatt

Teil 2 – Den Aufruf schreiben und Ergebnis checken

Gmünd → Stuttgart: 44,6 km

Gmünd → Gmünd: 0 km

Stuttgart → Gmünd: 44,6 km

(0,0) → (1, 0): 111,2 km (1 Breitengrad)

(0,0) → (0, 1): 111,2 km (1 Längengrad am Äquator)

(48.8, 9.79) → (48.8, 10.79): 73,2 km (1 Längengrad bei Gmünd)

Wochenaufgabe

1 - Sucht euch eine Sache im aktuellen Projekt `projektion.js`
Zum Beispiel: Die Beschriftungen der Orte überlappen manchmal

2 - Spezifiziert die Aufgabe - Wie könnt ihr das Ergebnis prüfen? - Schreibt den Prompt - prüft das Ergebnis

3 — Zwei Dateien: PROMPT.md und ENTSCHEIDUNGEN.md

Werkstatt

Teil 2 – Markieren

Grün: Verstehe ich, könnte ich selbst schreiben

Gelb: Ahne ich, könnte ich aber nicht erklären

Rot: Keine Ahnung

Viel Rot ist kein schlechtes Zeichen!